

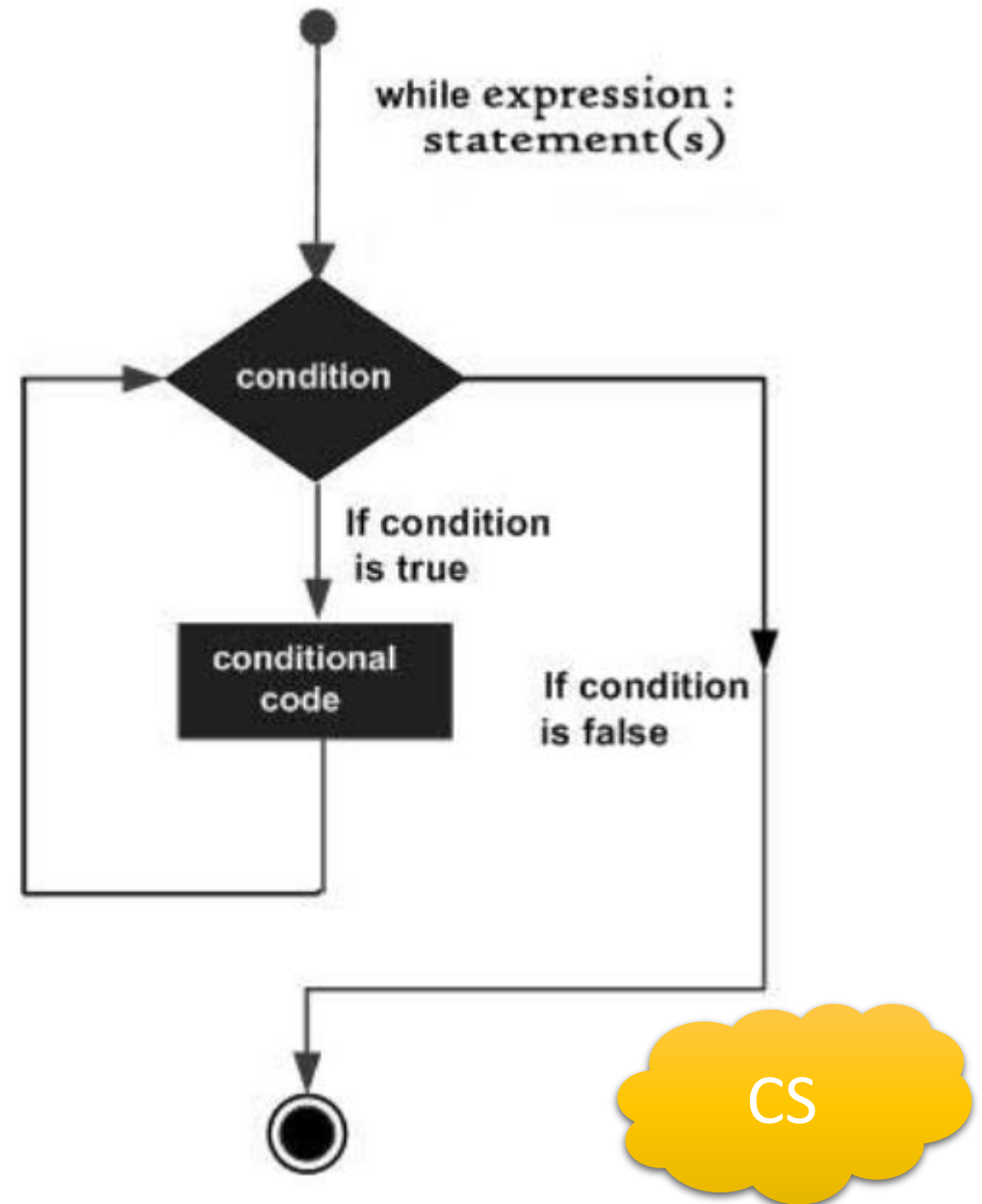
รหัสวิชา 2-233-225
ชื่อวิชา วิทยาศาสตร์ข้อมูลเบื้องต้น
(Introduction to Data Science)
สัปดาห์ที่ 5

CS

Repetition (while, for)

```
while expression:  
    statement(s)
```

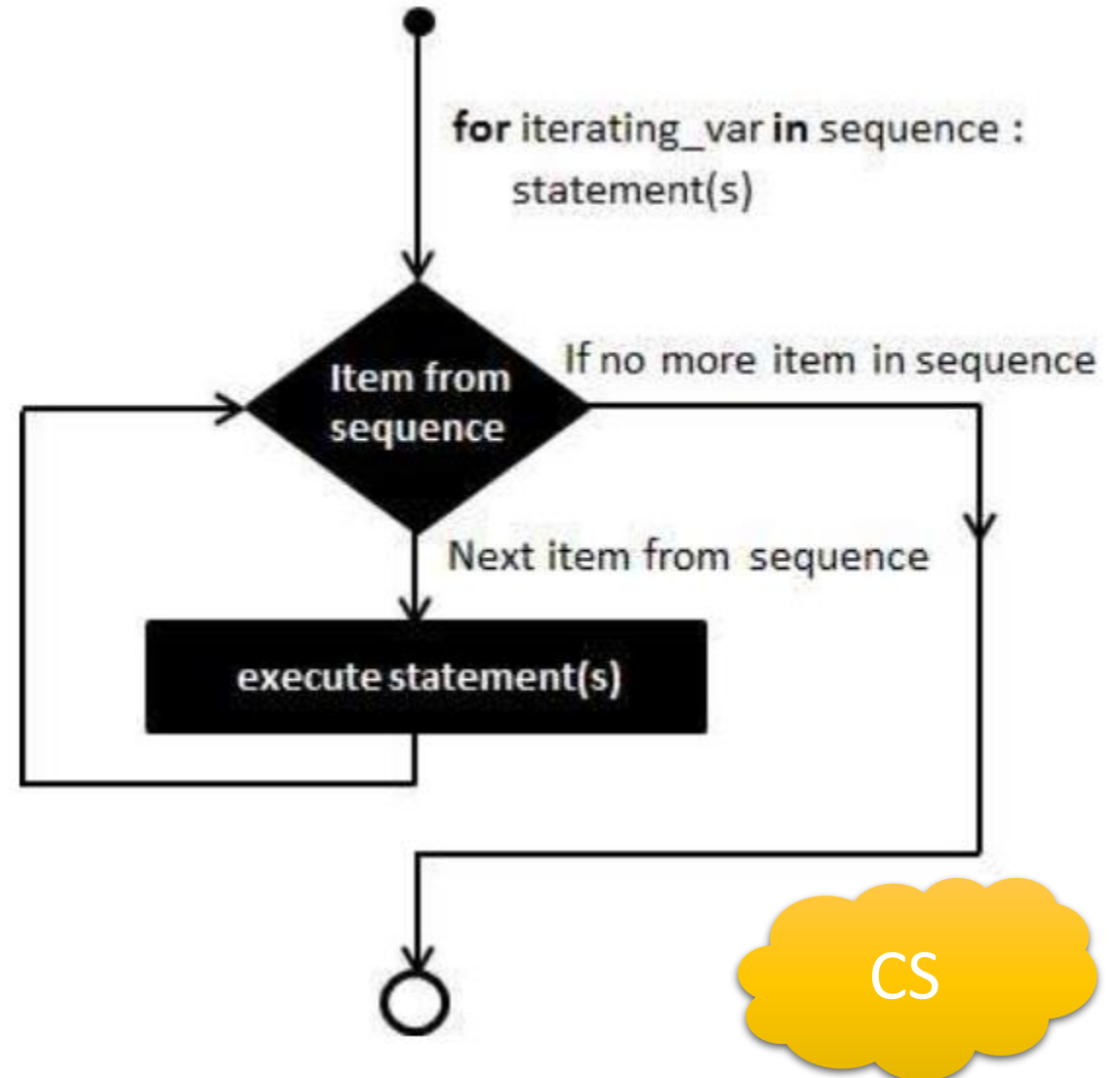
```
count = 0  
while (count < 9):  
    print 'The count is:', count  
    count = count + 1  
  
print "Good bye!"
```



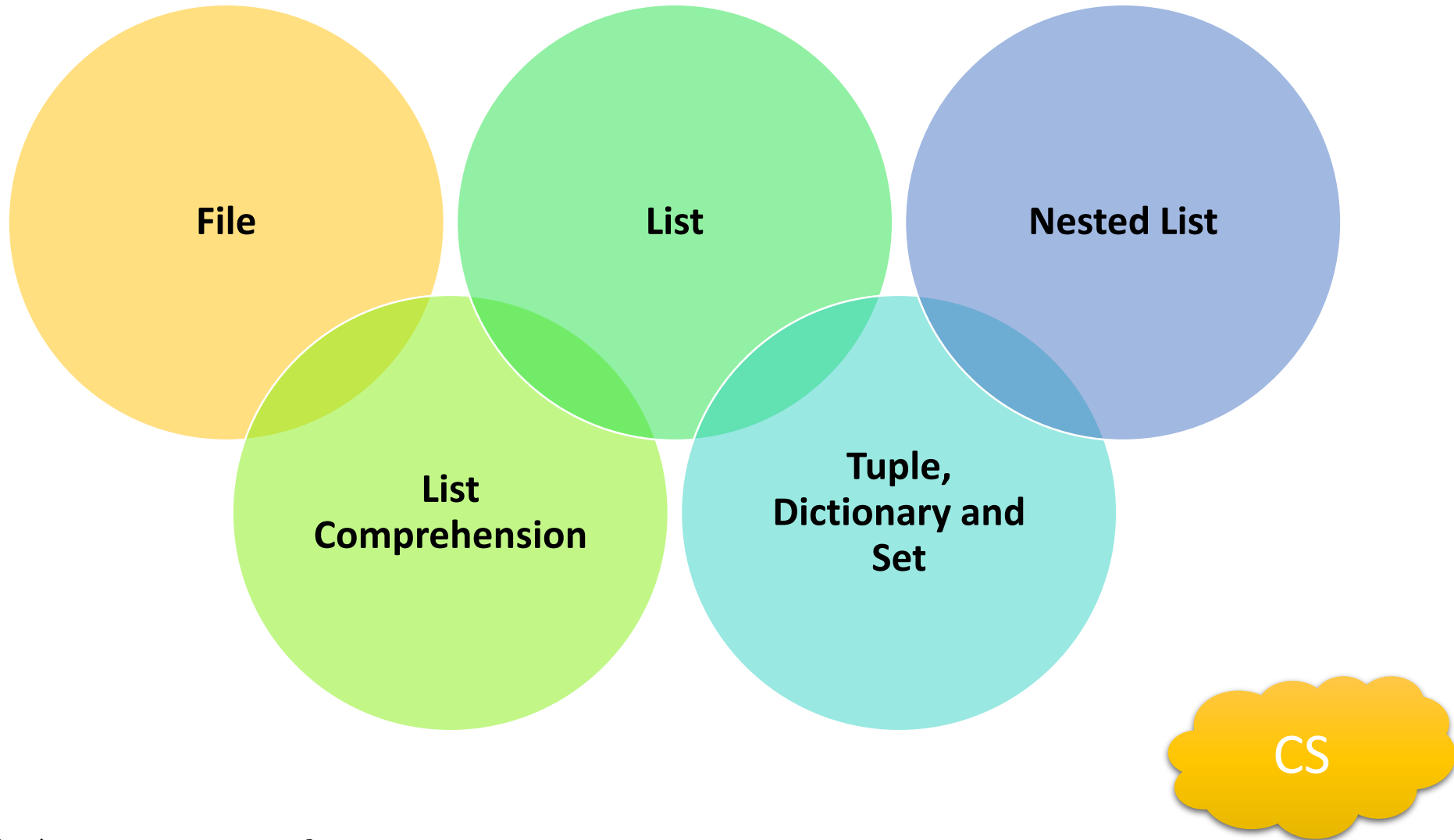
Repetition (while, for)

```
for iterating_var in sequence:  
    statements(s)
```

```
for letter in 'Python':    # First Example  
    print 'Current Letter :', letter  
  
fruits = ['banana', 'apple', 'mango']  
for fruit in fruits:       # Second Example  
    print 'Current fruit :', fruit  
  
print "Good bye!"
```



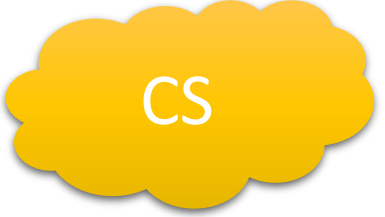
สัปดาห์ที่ 5



File

>>>Create and Write text file

```
In [115]: file = open('testfile.txt','w')
file.write('Hello World')
file.write('This is a new text file')
file.write('and this is another line.')
file.write('We can add data.')
file.close()
```

A yellow, stylized cloud shape with a soft shadow, containing the white text 'CS' in the center.

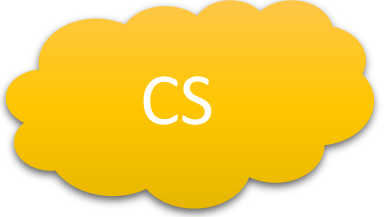
File

>>>Read text file

```
In [107]: f = open('test_file.txt')
while True:
    data = f.readline()
    if not data:
        break
    print(data)
f.close()
print('หมดข้อมูลในไฟล์ 1')
```

Test

หมดข้อมูลในไฟล์ 1



CS

List

Python List Methods

append() - Add an element to the end of the list

extend() - Add all elements of a list to the another list

insert() - Insert an item at the defined index

remove() - Removes an item from the list

pop() - Removes and returns an element at the given index

clear() - Removes all items from the list

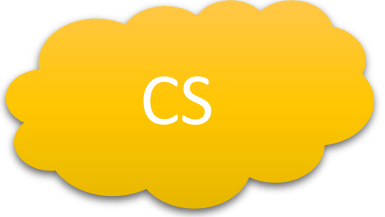
index() - Returns the index of the first matched item

count() - Returns the count of number of items passed as an argument

sort() - Sort items in a list in ascending order

reverse() - Reverse the order of items in the list

copy() - Returns a shallow copy of the list

A yellow cloud-shaped icon with the letters 'CS' in white text inside it.

List

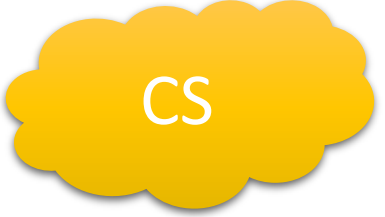
```
In [122]: my_list = [3, 8, 1, 6, 0, 8, 4]
# Output: 1
print(my_list.index(8))
# Output: 2
print(my_list.count(8))
my_list.sort()
# Output: [0, 1, 3, 4, 6, 8, 8]
print(my_list)
my_list.reverse()
# Output: [8, 8, 6, 4, 3, 1, 0]
print(my_list)
```

1

2

[0, 1, 3, 4, 6, 8, 8]

[8, 8, 6, 4, 3, 1, 0]



CS

Nested List

```
In [124]: matrix = []  
for i in range(5):  
    matrix.append([])  
    for j in range(5):  
        matrix[i].append(j)  
    print(matrix)
```

```
[[0]]  
[[0, 1]]  
[[0, 1, 2]]  
[[0, 1, 2, 3]]  
[[0, 1, 2, 3, 4]]  
[[0, 1, 2, 3, 4], [0]]  
[[0, 1, 2, 3, 4], [0, 1]]  
[[0, 1, 2, 3, 4], [0, 1, 2]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3]]  
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]
```

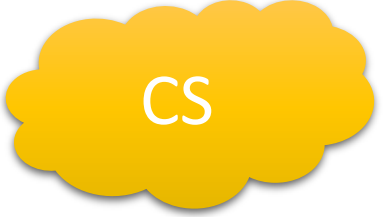
A yellow cloud-shaped logo with the letters 'CS' in white, representing Computer Science.

Nested List

```
In [125]: matrix = [[j for j in range(5)] for i in range(5)]
```

```
print(matrix)
```

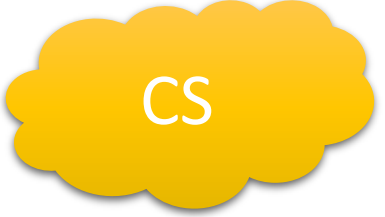
```
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]
```

A yellow, stylized cloud shape with a soft shadow, containing the white text "CS".

CS

Tuple, Dictionary and Set

```
In [127]: t = (1, 2, [3, 4])  
          t[2].append(5)  
  
          print(t)  # (1, 2, [3, 4, 5])  
  
(1, 2, [3, 4, 5])
```



CS

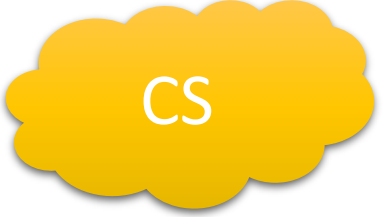
Tuple, Dictionary and Set

Tuple Unpacking

In [128]: `code, message = (404, 'Not Found')`

```
print(code) # 404  
print(message) # Not Found
```

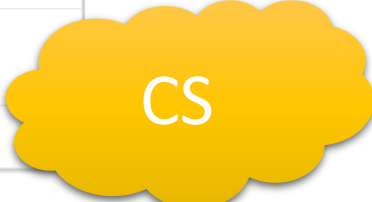
404
Not Found



CS

Tuple, Dictionary and Set

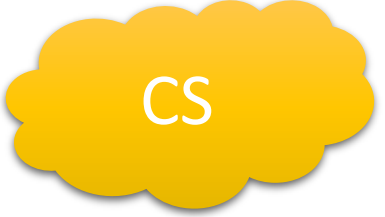
Python Dictionary Methods	
Method	Description
<code>clear()</code>	Remove all items form the dictionary.
<code>copy()</code>	Return a shallow copy of the dictionary.
<code>fromkeys(seq [, v])</code>	Return a new dictionary with keys from <code>seq</code> and value equal to <code>v</code> (defaults to <code>None</code>).
<code>get(key [, d])</code>	Return the value of <code>key</code> . If <code>key</code> doesnot exit, return <code>d</code> (defaults to <code>None</code>).
<code>items()</code>	Return a new view of the dictionary's items (key, value).
<code>keys()</code>	Return a new view of the dictionary's keys.
<code>pop(key [, d])</code>	Remove the item with <code>key</code> and return its value or <code>d</code> if <code>key</code> is not found. If <code>d</code> is not provided and <code>key</code> is not found, raises <code>KeyError</code> .
<code>popitem()</code>	Remove and return an arbitrary item (key, value). Raises <code>KeyError</code> if the dictionary is empty.
<code>setdefault(key [, d])</code>	If <code>key</code> is in the dictionary, return its value. If not, insert <code>key</code> with a value of <code>d</code> and return <code>d</code> (defaults to <code>None</code>).
<code>update([other])</code>	Update the dictionary with the key/value pairs from <code>other</code> , overwriting existing keys.
<code>values()</code>	Return a new view of the dictionary's values

A yellow cloud-shaped logo with the letters "CS" in white, positioned in the bottom right corner of the slide.

Tuple, Dictionary and Set

```
In [129]: d = { 'key1': 'Hello', 'key2': 'World' } # สร้าง dict ผ่าน {}  
          d['key1'] # Hello
```

```
Out[129]: 'Hello'
```

A yellow cloud-shaped logo with the letters 'CS' in white, located in the bottom right corner of the slide.

Tuple, Dictionary and Set

change or add elements in a dictionary

```
In [133]: my_dict = {'name': 'Jack', 'age': 26}

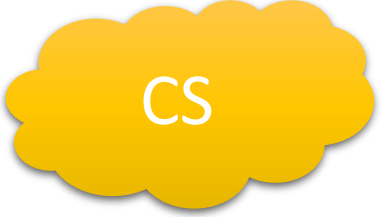
# update value
my_dict['age'] = 27

#Output: {'age': 27, 'name': 'Jack'}
print(my_dict)

# add item
my_dict['address'] = 'Downtown'

# Output: {'address': 'Downtown', 'age': 27, 'name': 'Jack'}
print(my_dict)

{'name': 'Jack', 'age': 27}
{'name': 'Jack', 'age': 27, 'address': 'Downtown'}
```

A yellow cloud-shaped logo with the letters 'CS' in white, located in the bottom right corner of the slide.

Tuple, Dictionary and Set

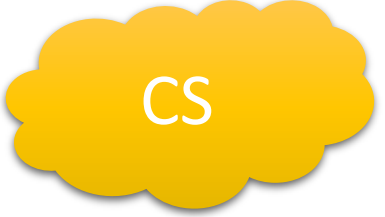
>>>Set (เซต)เป็นกลุ่มข้อมูล

```
In [135]: {1, 2}
```

```
Out[135]: {1, 2}
```

```
In [138]: ##แปลงข้อมูลเป็นเซต  
a = set([1,2,3])  
print(a,type(a))
```

```
{1, 2, 3} <class 'set'>
```



CS

Tuple, Dictionary and Set

>>>Set (เซต)เป็นกลุ่มข้อมูล

```
In [139]: ##แปลงจาก tuples เป็นเซต  
aa =(1, 2, 3, 4, 5)  
c = list(aa)  
set(c)
```

```
Out[139]: {1, 2, 3, 4, 5}
```



CS

Tuple, Dictionary and Set

>>>Set (เซต)เป็นกลุ่มข้อมูล

In [140]:

```
##แปลงจากข้อมูลอื่นเป็นเซต  
a = "12345"  
b = set(a)  
print(b)
```

```
{'2', '3', '5', '4', '1'}
```



CS

Tuple, Dictionary and Set

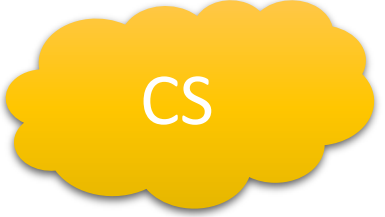
>>>Set (เซต) เป็นกลุ่มข้อมูล

```
In [142]: ## เซตข้อมูลที่อยู่ในเซต
a_set = {10, 11, 12, 13, 14, 15, 16, 17, 18, 19}
1 in a_set
```

Out[142]: False

```
In [143]: 10 in a_set
```

Out[143]: True



CS

Tuple, Dictionary and Set

>>>Set (เซต)เป็นกลุ่มข้อมูล

```
In [145]: ##เพิ่มข้อมูลเข้าไปในเซต  
b = {1, 2, 3}  
b.update([4,5])  
print(b)
```

```
{1, 2, 3, 4, 5}
```



CS

Tuple, Dictionary and Set

>>>Set (เซต)เป็นกลุ่มข้อมูล

In [146]:

```
##ลบสมาชิกออกจากเซต  
b = {1, 2, 3, 4, 5}  
b.discard(4)  
print(b)
```

```
{1, 2, 3, 5}
```



CS

การดำเนินการเซต

ยูเนียน (Set Union)

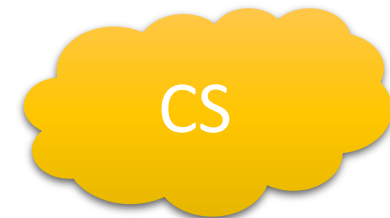
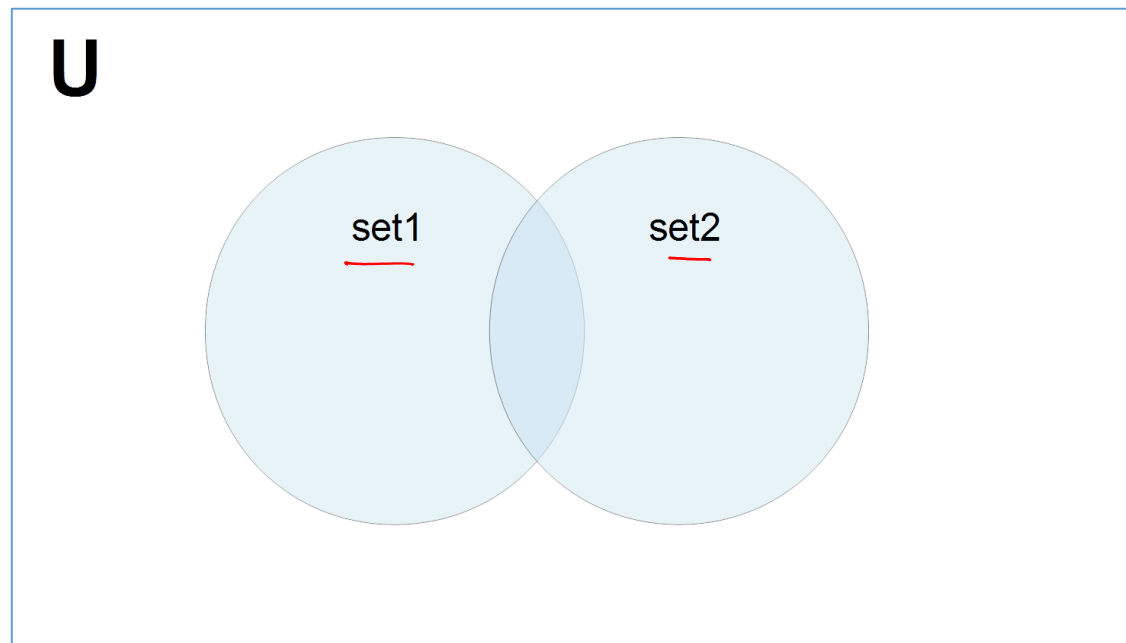
อินเตอร์เซกชัน (Set Intersection)

ผลต่างของเซต (Set Difference)

ผลต่างสมมาตรของเซต (Symmetric difference)

CS

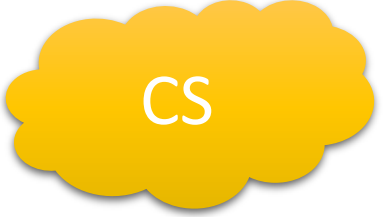
✓ ยูเนียน (Set Union)



✓ ยูเนียน (Set Union)

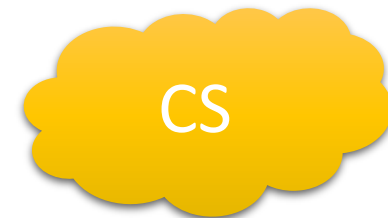
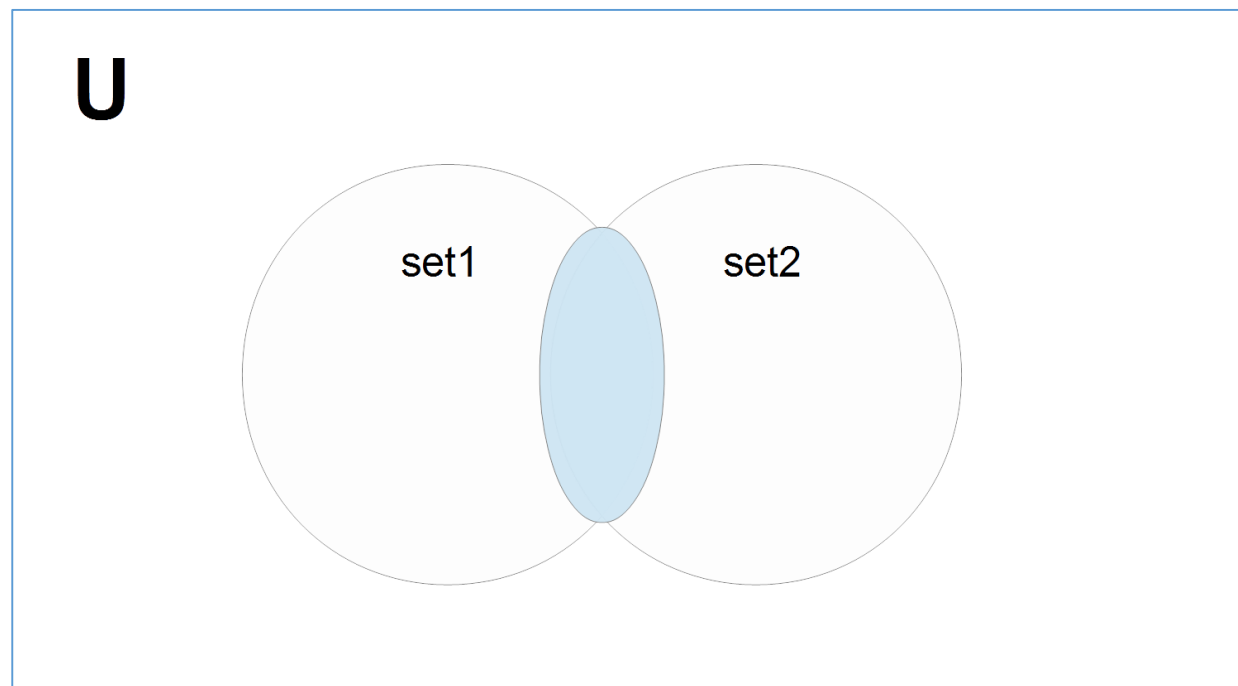
```
In [147]: set1 = {11, 12, 13, 14, 15}
          set2 = {14, 15, 16, 17, 18}
          set1 | set2
```

```
Out[147]: {11, 12, 13, 14, 15, 16, 17, 18}
```



CS

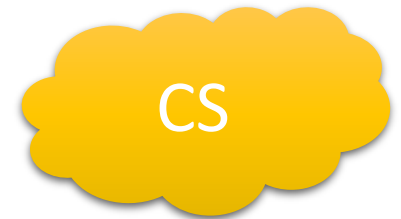
✓ อินเตอร์เซกชัน (Set Intersection)



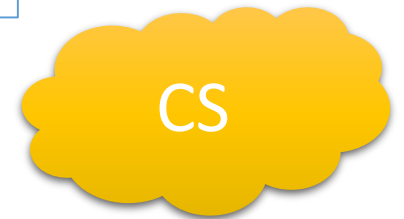
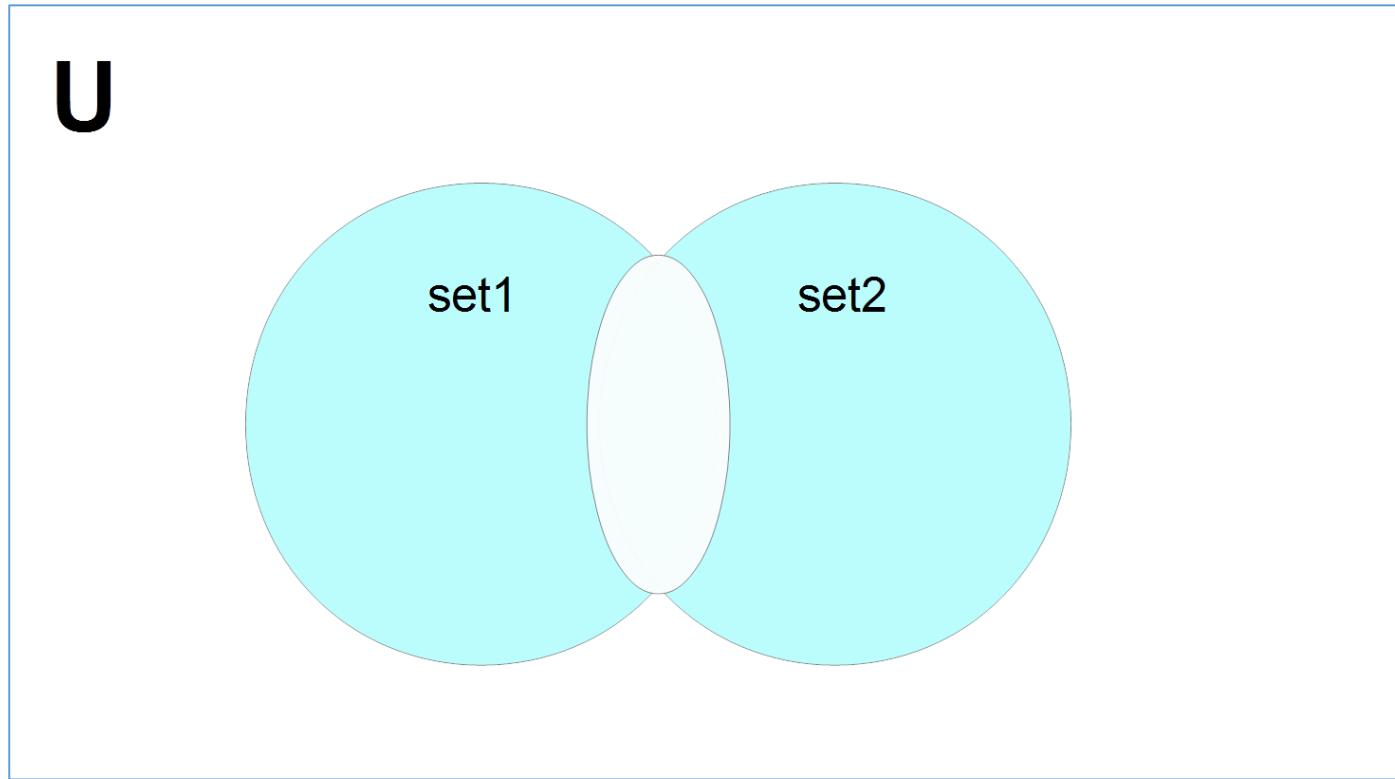
✓ อินเตอร์เซกชัน (Set Intersection)

```
In [148]: set1 & set2
```

```
Out[148]: {14, 15}
```



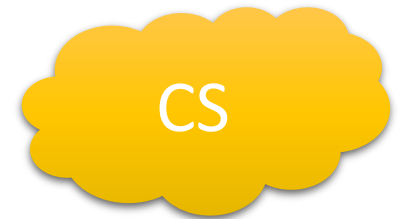
ผลต่างของเซต (Set Difference)



ผลต่างของเซต (Set Difference)

```
In [149]: set1 - set2
```

```
Out[149]: {11, 12, 13}
```



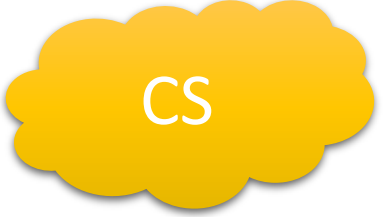
ผลต่างสมมาตรของเซต (Symmetric difference)

```
In [150]: set1 ^ set2
```

```
Out[150]: {11, 12, 13, 16, 17, 18}
```

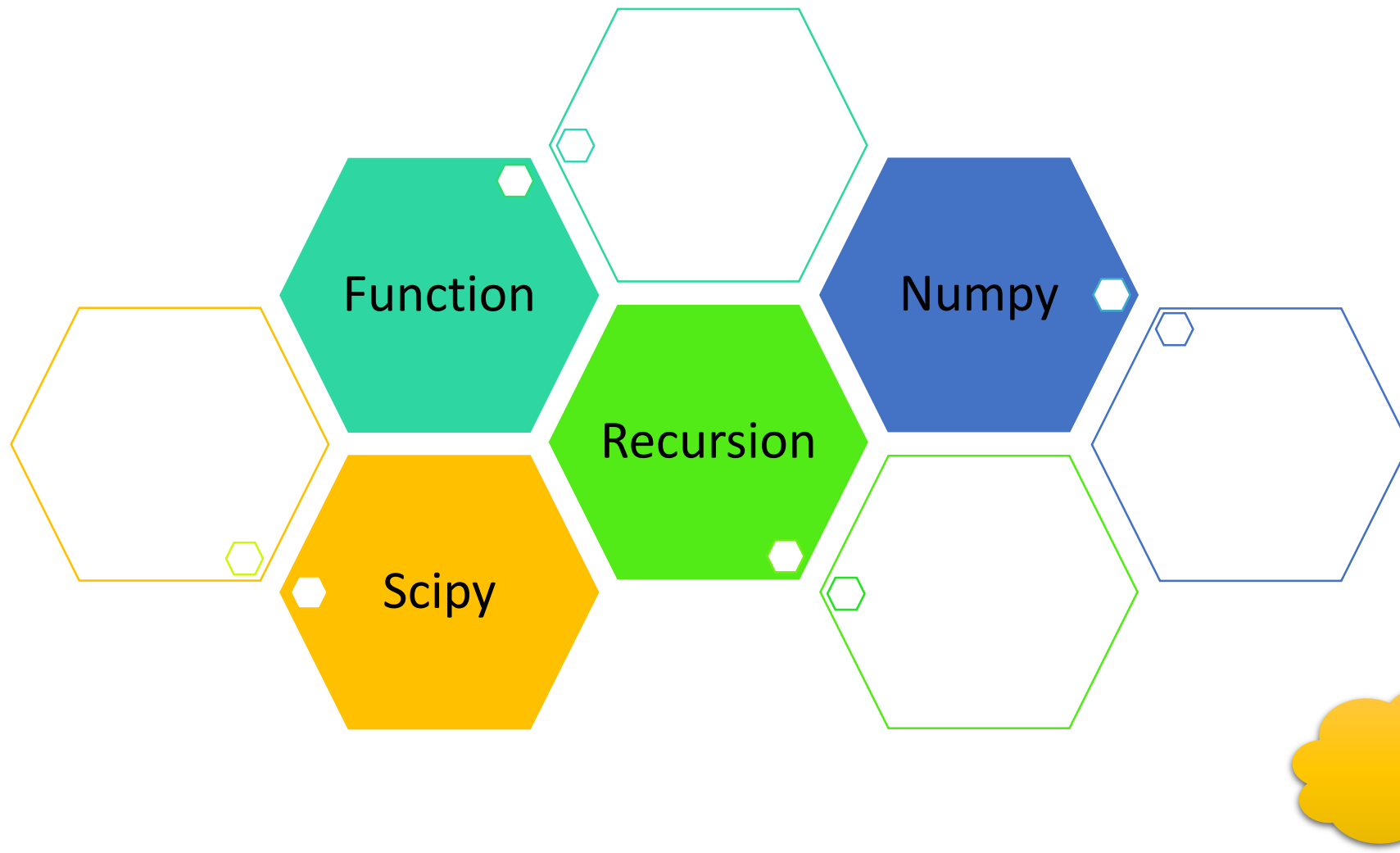
```
In [151]: set1.symmetric_difference(set2)
```

```
Out[151]: {11, 12, 13, 16, 17, 18}
```



CS

สัปดาห์ที่ 6-7



Function

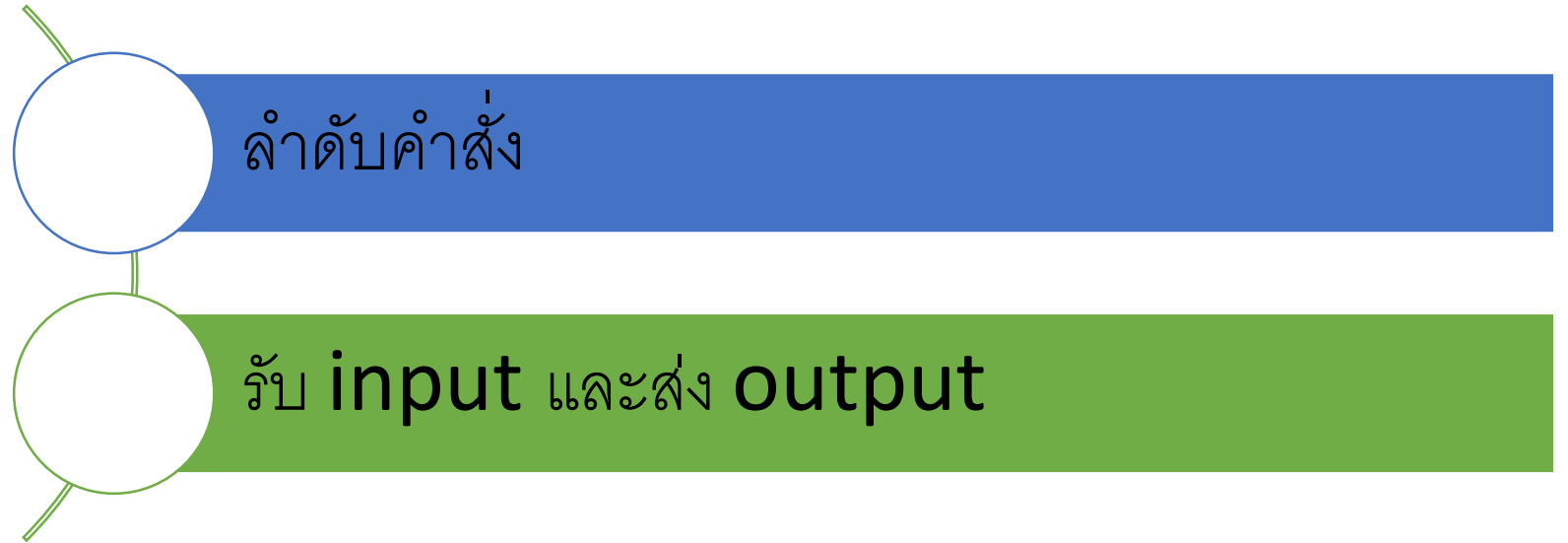
เขียนโค้ดเป็นลำดับคำสั่ง

Start to Finish

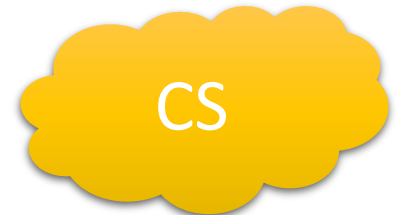
Call function

CS

Function

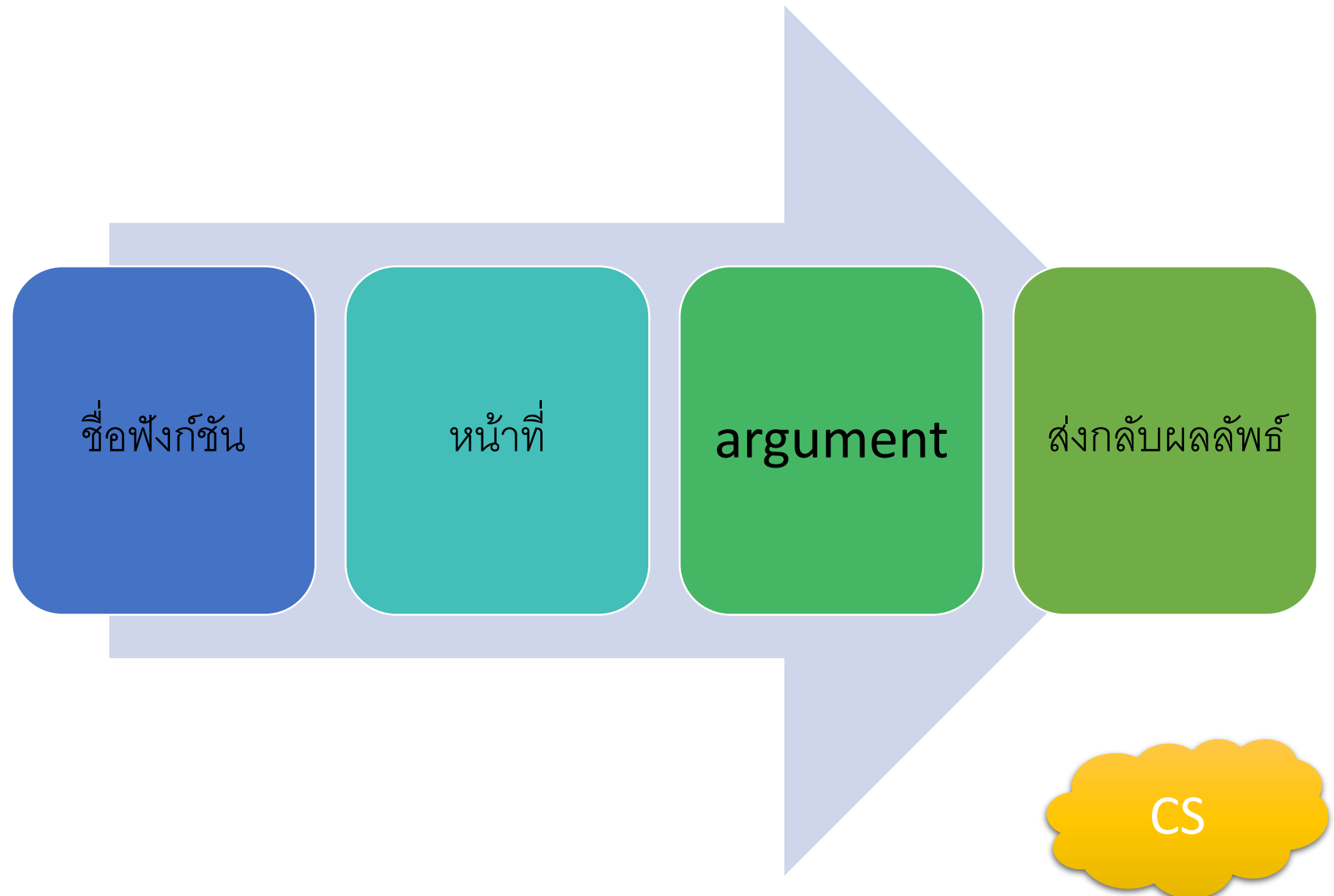


```
x = len("python")  
print(x)
```



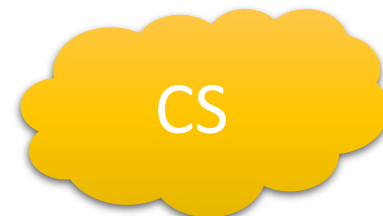
Function

- Abstraction



Function

- Reusability



Function

- Easy to maintenance

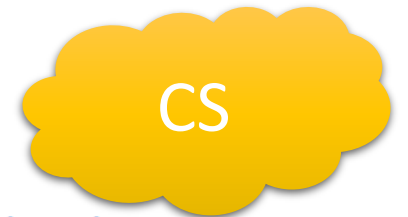
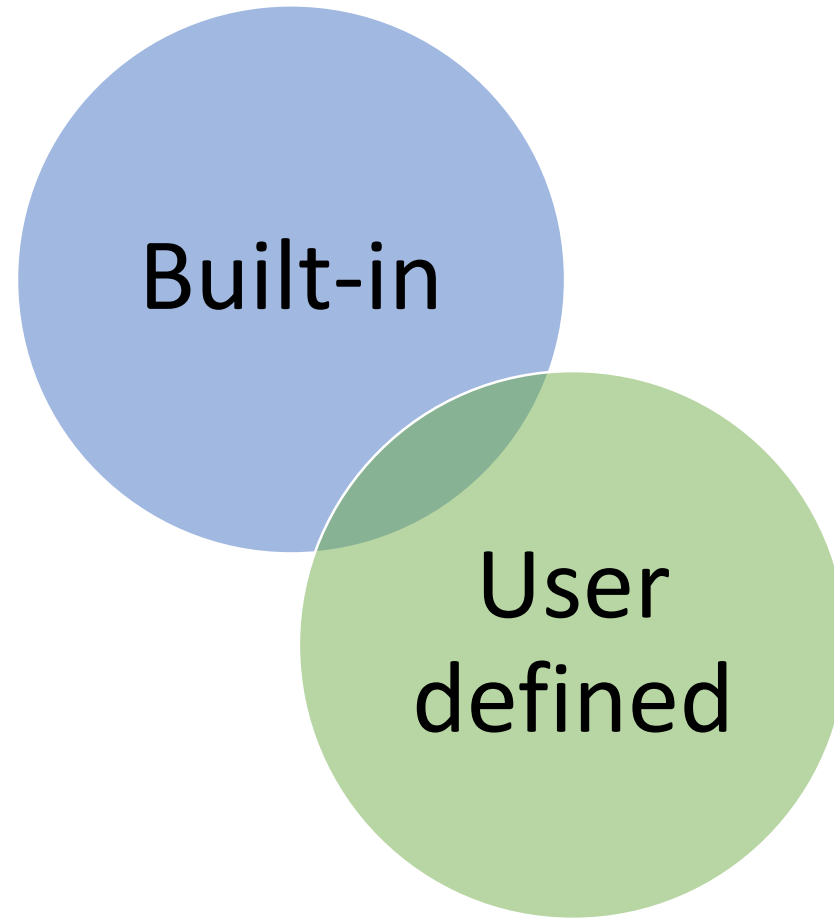
```
data = "python"
if len(data) > 4:
    print("not ok")
else
    print("ok")
```

```
data = "python"
i=1
While True:
    try:
        data[i]
        i=i+1
    except IndexError:
        break
if i > 4:
    print("not ok")
else
    print("ok")
```



CS

Function



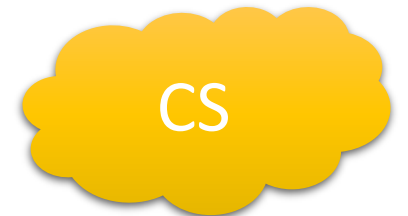
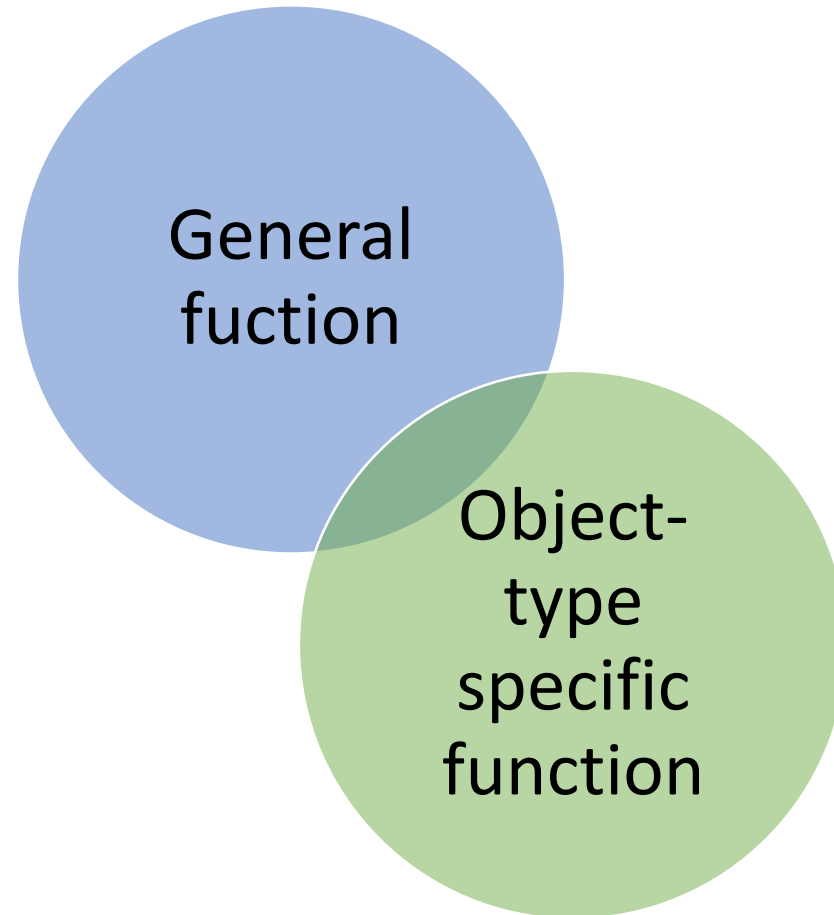
Built-in

Built-in functions				
abs()	dict()	help()	min()	setattr()
all()	dir()	hex()	next()	slice()
any()	divmod()	id()	object()	sorted()
ascii()	enumerate()	input()	oct()	staticmethod()
bin()	eval()	int()	open()	str()
bool()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
complete()	globals()	map()	reversed()	_import()
complex()	hasattr()	max()	round()	
delattr()	hash()	memoryview()	set()	

CS

http://www.cs.science.cmu.ac.th/course/204101/lib/exe/fetch.php?media=w07-lec_basic-function.pdf

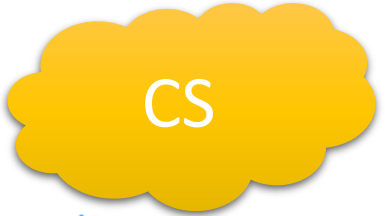
Built-in- 2 types



Function specification

- จำนวน **argument** ที่ต้องส่งให้ฟังก์ชัน
- ลำดับของ **argument** ที่ต้องส่งให้ฟังก์ชัน

```
>>> len("Python 3")  
8
```

A yellow cloud-shaped logo with the letters "CS" in white.

Function-help

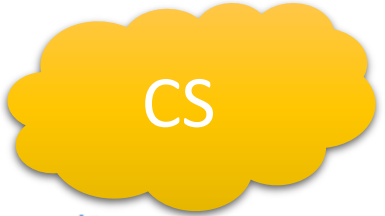
```
>>>  
>>> help(len)
```

```
Help on built-in function len in module builtins:
```

```
len(...)  
    len(object)
```

```
    Return the number of items of a sequence  
    or collection.
```

```
~  
~
```

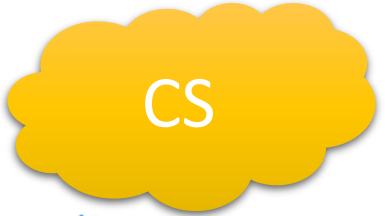
A yellow, stylized cloud shape with a white outline, containing the letters 'CS' in white. It is positioned in the lower right area of the slide.

Function

- ฟังก์ชันมีการส่งค่ากลับ >> fruitful function
- ฟังก์ชันที่ไม่มีการส่งค่ากลับ >> void function

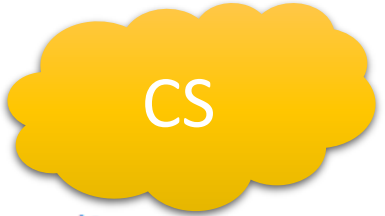
```
>>> a = print('b')
b
>>> print(a)
None

>>> a = log(2)
>>> print(a)
>>> 0.6931471805599453
```

A yellow cloud-shaped logo with the letters 'CS' in white.

Function-module

- นิยามค่าคงที่ (constants)
- ฟังก์ชันต่าง ๆ
 - A 'math' module contains a set of mathematical functions.
 - $\exp()$, $\log()$,
 - It also defines some useful constants in mathematics.
 - π



CS

Function-module

- เรียกใช้ฟังก์ชันใน module

from module_name **import** function_name

```
>>> from math import log  
>>> log(2)
```

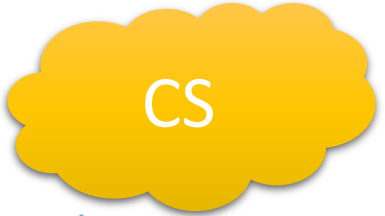
- Import ทุกฟังก์ชัน

```
>>> from math import *
```

Function-module

- เปลี่ยนชื่อฟังก์ชันใน module

```
>>> from math import log as mylog  
>>> mylog(2)
```

A yellow cloud-shaped logo with the letters "CS" in white.

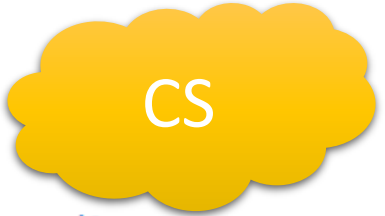
Function-Making use of the return values

- Storing

```
A1= exp(1)  
A2 = exp(2)  
A3=A1+A2
```

- On the fly

```
A3=exp(1)+exp(2)
```

A yellow cloud-shaped logo with the letters "CS" in white.

Function-Functions composition

```
A =  $\sqrt{\log(10)}$ 
```

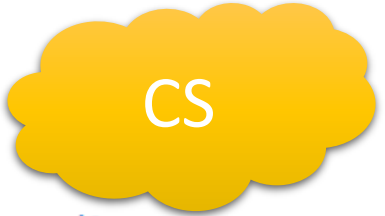
```
A1= log(10)
```

```
A2 = sqrt(A1)
```

```
Print(A2)
```

```
A1= sqrt(log(10))
```

```
Print(A1)
```

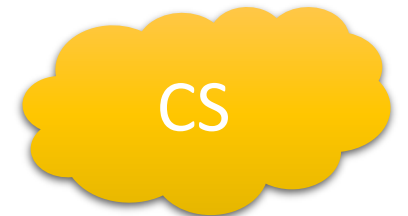
A yellow, stylized cloud shape with a soft drop shadow. Inside the cloud, the letters "CS" are written in a white, sans-serif font.

การสร้างฟังก์ชัน

```
def function_name(args...):  
    # statements
```

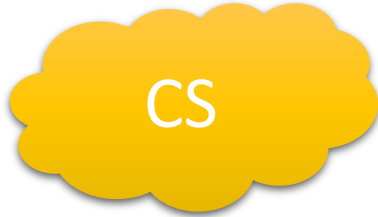
```
def function_name(args...):  
    # statements  
    return value
```

<http://marcuscode.com/lang/python/functions>



ตัวอย่างการประกาศและใช้งานฟังก์ชันในภาษา Python

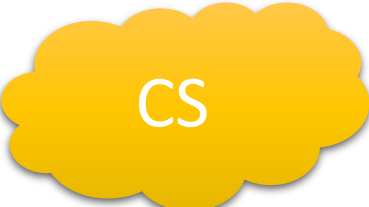
```
In [7]: def hello(name):  
        print('Hello %s' % name)
```

A yellow cloud-shaped icon with a white border, containing the letters 'CS' in white.

<http://marcuscode.com/lang/python/functions>

ตัวอย่างการประกาศและใช้งานฟังก์ชันในภาษา Python

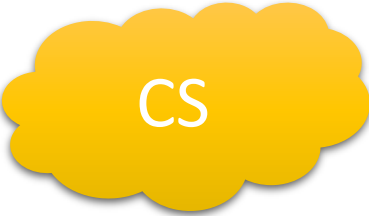
```
In [8]: def count_vowel(str):  
        vowel = 0  
        for c in str:  
            if c in ('A', 'E', 'I', 'O', 'U', 'a', 'e', 'i', 'o', 'u'):  
                vowel = vowel + 1  
        return vowel
```

CS

<http://marcuscode.com/lang/python/functions>

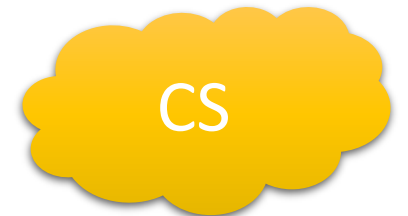
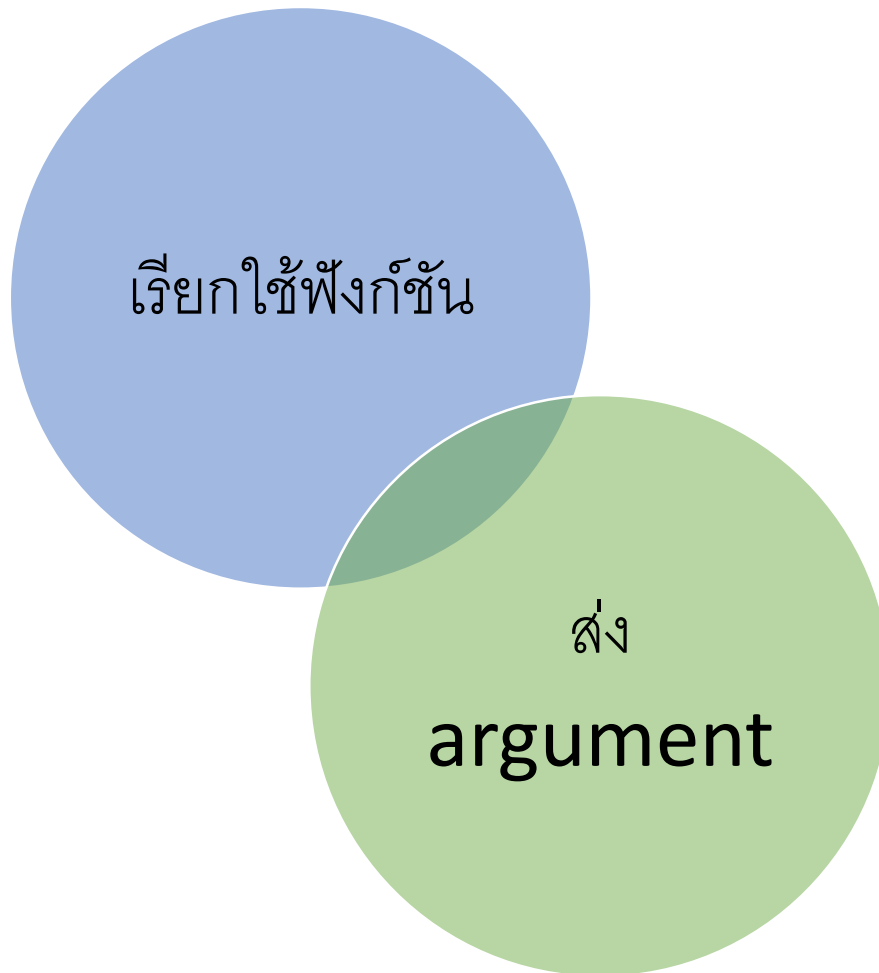
ตัวอย่างการประกาศและใช้งานฟังก์ชันในภาษา Python

```
In [9]: def area(width, height):  
        c = width * height  
        return c
```

CS

<http://marcuscode.com/lang/python/functions>

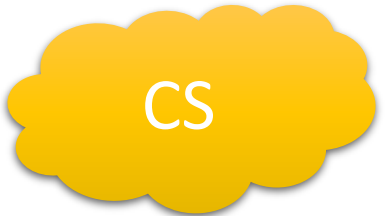
ตัวอย่างการประกาศและใช้งานฟังก์ชันในภาษา Python



ตัวอย่างการประกาศและใช้งานฟังก์ชันในภาษา Python

```
In [12]: # calling functions
hello('Danny')
hello('Mateo')
print('Vowel in string = %d' % count_vowel('marcuscode.com'))
print('Vowel in string = %d' % count_vowel('Python'))
print('Area = %d' % area(8, 4))
```

```
Hello Danny
Hello Mateo
Vowel in string = 5
Vowel in string = 1
Area = 32
```

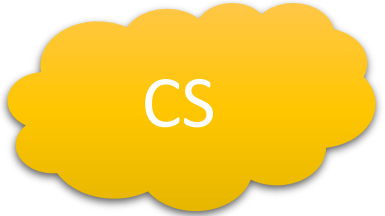


CS

Default Argument Values

- กำหนด Default Argument

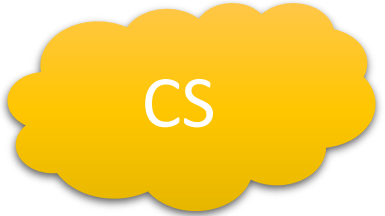
```
In [17]: def show_info(name, salary = 15000, lang = "Python"):
          print('Name: %s' % name)
          print('Salary: %d' % salary)
          print('Language: %s' % lang)
          print()
```

A yellow cloud-shaped logo with the letters "CS" in white text inside it.

Default Argument Values-ใช้งานฟังก์ชันในภาษา Python

```
In [19]: # calling function  
show_info('Ploy')
```

```
Name: Ploy  
Salary: 15000  
Language: Python
```



CS

Keyword Arguments

- เรียกใช้งานฟังก์ชันในรูปแบบของ **Keyword Argument** โดยการใช้ ชื่อของพารามิเตอร์สำหรับส่ง อาร์กิวเมนต์

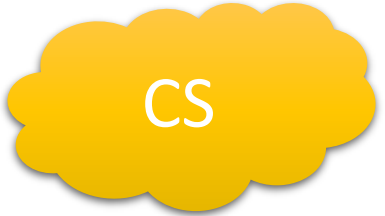
```
In [1]: def create_button(id, color = '#ffffff', text = 'Button', size = 16):  
        print('Button ID: %d' % id)  
        print('Attributes:')  
        print('Color: %s' % color)  
        print('Text: %s' % text)  
        print('Size: %d px' % size)  
        print()  
  
        create_button(10)  
        create_button(11, color = '#4286f4', text = 'Sign up')  
        create_button(id = 12, color = '#323f54', size = 24)  
        create_button(color = '#1cb22b', text = 'Log in', size = 32, id = 13)
```

CS

Lambda Expressions

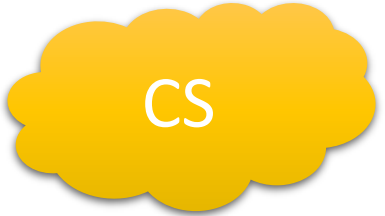
- Lambda Expressions คือ anonymous function >>>lambda

```
f = lambda x: x + 1
print(f(2))
print(f(8))
g = lambda a, b: (a + b) / 2
print(g(3, 5))
print(g(10, 33))
```

A yellow cloud-shaped logo with the letters "CS" in white.

Lambda Expressions

```
In [4]: def make_incrementor(n):  
        return lambda x: x + n  
f = make_incrementor(13)  
print(f(0))  
print(f(1))  
print(f(5))
```

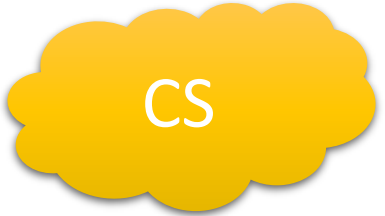
A yellow, stylized cloud shape with a soft shadow, containing the white text 'CS'.

Lambda Expressions

Lambda Expressions>>>built-in function

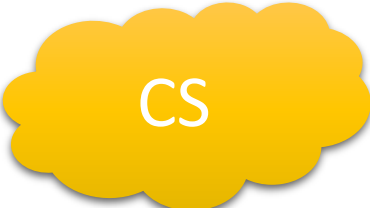
```
In [5]: numbers = [2, 15, 5, 7, 10, 3, 28, 30]
print(list(filter(lambda x: x % 5 == 0, numbers)))
print(list(map(lambda x: x * 2, numbers)))

[15, 5, 10, 30]
[4, 30, 10, 14, 20, 6, 56, 60]
```

A yellow cloud-shaped logo with the letters 'CS' in white, located in the bottom right corner of the slide.

Recursion

def ฟังก์ชัน(พารามิเตอร์):
if < เงื่อนไขที่ผลให้ยุติการเรียกซ้ำ (จุดวกกลับ หรือ **base case**) >:
 ส่งคำตอบคืน
else:
 แยกปัญหาเป็นประเด็นย่อย ด้วยการเรียกซ้ำ (**Recursive**)



CS

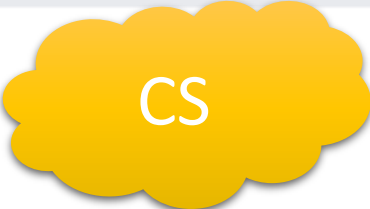
<https://python3.wannaphong.com/2017/05/recursive-function-python.html>

Recursion

```
n! = n*(n-1)! if n < 0
```

เอานามาหา 5!

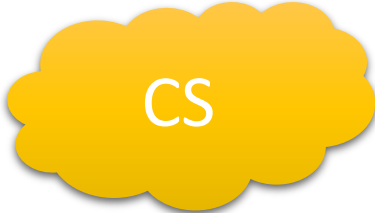
```
5! = 5*4*3*2*1 = 5*4!  
4! = 4*3*2*1 = 4*3!  
3! = 3*2*1 = 3*2!  
2! = 2*1 = 2*1!  
1! = 1 = 1  
0! = 1 นี่คือการจุดวกกลับ หรือ base case
```

A yellow cloud-shaped logo with the letters "CS" in white.

<https://python3.wannaphong.com/2017/05/recursive-function-python.html>

Recursion

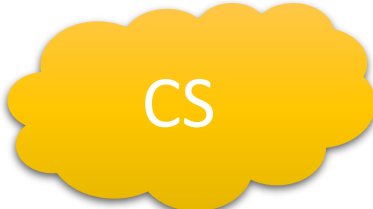
```
In [7]: def factorial(num):  
        if num==0:  
            return 1  
        else:  
            return num * factorial(num-1)  
  
        print(factorial(5))  
        print(factorial(8))
```

A yellow cloud-shaped logo with the letters 'CS' in white, located in the bottom right corner of the slide.

<https://python3.wannaphong.com/2017/05/recursive-function-python.html>

NumPy

- Numpy คือ core library สำหรับการคำนวณของ Python ซึ่งเก็บข้อมูลในรูปแบบของ array เพื่อนำไปประมวลผล

A yellow cloud-shaped logo with the letters "CS" in white text inside it.

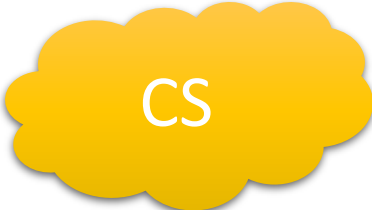
<http://cs231n.github.io/python-numpy-tutorial/#numpy>

NumPy

- Array of Numpy เป็นตารางการเก็บข้อมูลและเก็บเป็น index

```
In [1]: import numpy as np
a = np.array([1, 2, 3])    # Create a rank 1 array
print(type(a))            # Prints "<class 'numpy.ndarray'>"
print(a.shape)            # Prints "(3,)"
print(a[0], a[1], a[2])   # Prints "1 2 3"
a[0] = 5                  # Change an element of the array
print(a)                  # Prints "[5, 2, 3]"

b = np.array([[1,2,3],[4,5,6]]) # Create a rank 2 array
print(b.shape)            # Prints "(2, 3)"
print(b[0, 0], b[0, 1], b[1, 0]) # Prints "1 2 4"
```

CS

<http://cs231n.github.io/python-numpy-tutorial/#numpy>

NumPy

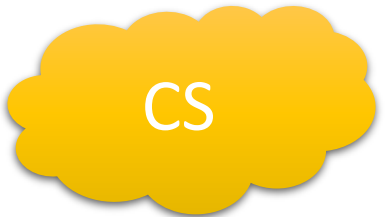
```
In [3]: x = np.arange(10)  
        y = x + 10x
```

```
Out[3]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [4]: y = x + 10
```

```
In [5]: y
```

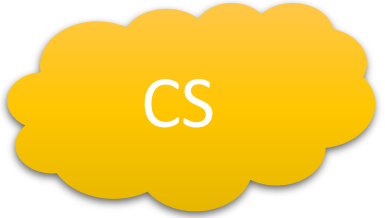
```
Out[5]: array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

A yellow, stylized cloud shape with a white outline, containing the letters 'CS' in white.

<http://cs231n.github.io/python-numpy-tutorial/#numpy>

Scipy

- **Numpy** จัดเตรียมอาร์เรย์หลายมิติประสิทธิภาพสูงและเครื่องมือพื้นฐานในการคำนวณและจัดการกับอาร์เรย์เหล่านี้ **SciPy** สร้างขึ้นจากสิ่งนี้และให้ฟังก์ชันจำนวนมากที่ทำงานในอาร์เรย์ **numpy** และมีประโยชน์สำหรับการทำงานทางวิทยาศาสตร์และวิศวกรรมประเภทต่าง ๆ

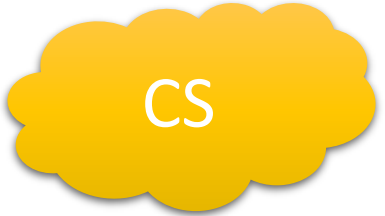
A yellow cloud-shaped icon with a white border, containing the letters 'CS' in white.

<http://cs231n.github.io/python-numpy-tutorial/#numpy>

Scipy

- Image operations

SciPy ให้ฟังก์ชันพื้นฐานบางอย่างเพื่อทำงานกับภาพ ตัวอย่างเช่นมีฟังก์ชันในการอ่านภาพจากดิสก์ลงในอาร์เรย์ที่มี **numpy** เพื่อเขียนอาร์เรย์ **numpy** ไปยังดิสก์เป็นภาพและเพื่อปรับขนาดภาพ

A yellow cloud-shaped logo with the letters 'CS' inside in white.

<http://cs231n.github.io/python-numpy-tutorial/#numpy>

Scipy imread

```
In [13]: import numpy as np
import scipy
from matplotlib.pyplot import imread
img = imread('cat.jpg')
print(img.dtype, img.shape)
```

```
uint8 (1280, 1920, 3)
```

A yellow cloud-shaped logo with the letters 'CS' in white.

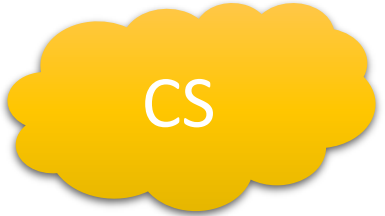
<http://cs231n.github.io/python-numpy-tutorial/#numpy>

Scipy **imsave**

```
In [31]: import numpy as np
import scipy
import matplotlib.pyplot as plt
from matplotlib.pyplot import imread

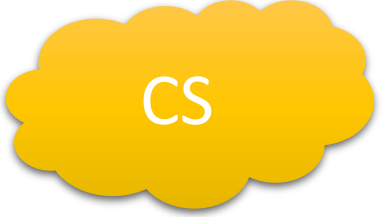
img = imread('cat.jpg')
print(img.dtype, img.shape)
plt.imsave('image_new.jpg',img )

uint8 (1280, 1920, 3)
```

A yellow, stylized cloud shape with a white outline, containing the letters 'CS' in white.

อ้างอิง

- Jesús Rogel-Salazar. DATA SCIENCE AND ANALYTICS WITH PYTHON CRC Press Taylor & Francis Group, 2017.
- Jake VanderPlas. Python Data Science Handbook . O'REILLY, 2017.



CS